



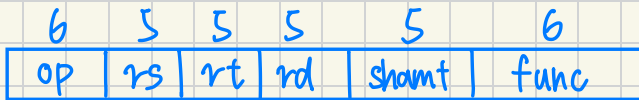
考古:

1、

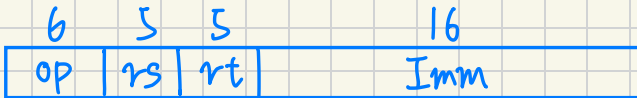
- (1) - 資料傳輸: lw, sw (2) ll (load and link) & sc (store conditional)
- 算術: add, sub
- 邏輯: and, or
- 條件分支: bne, beq

2、

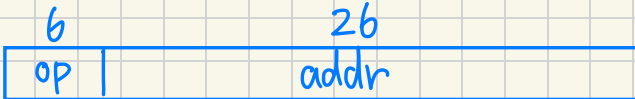
R format:



I format:



J format:



3. add (func: 32) R17, R18, R19
rd rs rt

R-format:

op	(R18) rs	(R19) rt	(R17) rd	shamt	func
000000	10010	10011	10001	00000	100000

名稱	暫存器號碼	用法	呼叫時是否保存
\$zero	0	常數值 0	不適用
\$v0-\$v1	2-3	運算結果與式子的數值	不保存
\$a0-\$a3	4-7	參數	不保存
\$t0-\$t7	8-15	暫時值	不保存
\$s0-\$s7	16-23	被保存的值	保存
\$t8-\$t9	24-25	更多暫時值	不保存
\$gp	28	全域指標	保存
\$sp	29	堆疊指標	保存
\$fp	30	程序框指標	保存
\$ra	31	返回位址	保存

4. De-assemble MIPS assembly language :

op	rs	rt	rd	shamt	func
0000 00	0 000	0000	1000 0000	00 0	0000
0	R16	R16	R16	0	32
(R-type)	(\$S0)	(\$S0)	(\$S0)		(add)

add \$S0, \$S0, \$S0

add \$S1, \$S2, \$S3
rd rs rt
\$S1 ← \$S2 + \$S3

5.

(1)

bne, beq
slt, sltu
slti, sltiu

(2)

比大小的指令較為複雜，
改先實現其他較簡單的指令，
並以組合指令來實現 blt 的功能

(3) branch less then

slt \$t1, \$S1, \$S2
bne \$t1, \$zero, L
L:
.....

6.

addi \$t1, \$0, 10 $i = 10$
addi \$S2, \$0, \$0 $B = 0$
LOOP: slt \$t2, \$0, \$t1 if ($0 < i$), temp = 1
beq \$t2, \$0, DONE if temp = 0, Jump
subi \$t1, \$t1, 1 $i = i - 1$
addi \$S2, \$S2, 2 $B = B + 2$

DONE:

.....

\$S1, \$S2, \$t1, \$t2
A B i temp

int i=10, B=0;
while(i>0){

i--;
B+=2;

}
for(int i=10; i>0; i--){
B+=2;
}

B的最終值: 20

7. Use MIPS Implement below C Program

```
int fib(int n){  
    if(n==0) return 0;  
    else if (n==1) return 1;  
    else return fib(n-1)+fib(n-2);  
}
```

Hard!

```
fib: addi $sp, $sp, -8  
     sw $ra, 4($sp)  
     sw $a0, 0($sp)  
     sli $t0, $a0, 2  
     bne $t0, $zero, base  
     addi $a0, $a0, -1  
     jal fib  
     sw $v0, 4($sp)  
     lw $a0, 0($sp)  
     addi $a0, $a0, -2  
     jal fib  
     lw $t1, 4($sp)  
     add $v0, $v0, $t1  
     j rtn
```

```
base:  
     add $v0, $a0, $zero  
  
rtn:  
     lw $ra, 4($sp)  
     addi $sp, $sp, 8  
     jr $ra
```

課本題

2.3 f.g.h.i.j $\rightarrow$ \$S0, \$S1, \$S2, \$S3, \$S4
A.B. \$S6, \$S7

sub \$t0, \$S3, \$S4

sll \$t0, \$t0, 2

add \$t0, \$t0, \$S6

lw \$t1, 0(\$t0)

sw \$t1, 32(\$S7)

2.4

$$\begin{aligned} B[g] &= A[f+1] + f \\ &= A[f+1] + A[f] \end{aligned}$$

2.5

sll \$t0, \$S0, 2

add \$t0, \$t0, \$S6

lw \$t1, 0(\$t0)

lw \$t2, 4(\$t1)

add \$t3, \$t1, \$t2

sll \$t1, \$S1, 2

add \$t1, \$S7, \$t1

sw \$t3, 0(\$t1)

2.9

```

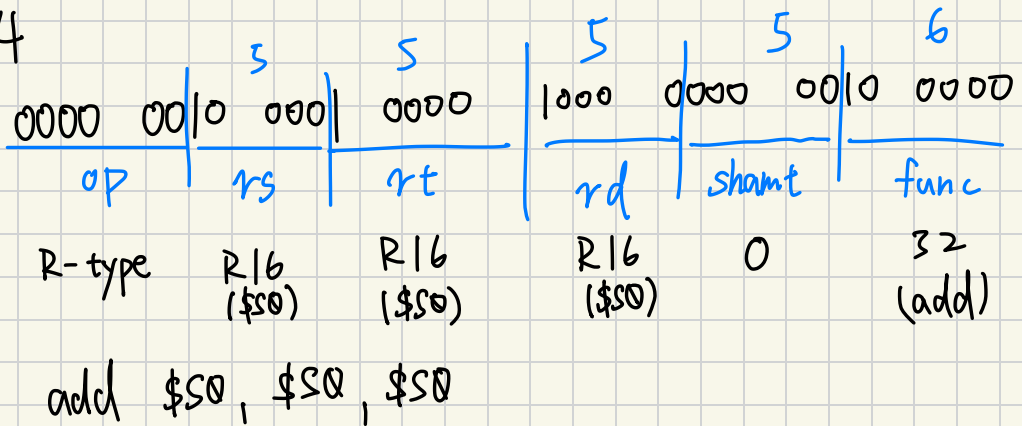
sll $s3, $s3, 2
sll $s4, $s4, 2
add $s3, $s3, $s6
add $s4, $s4, $s6
lw $t0, 0($s3)
lw $t1, 0($s4)
add $t2, $t0, $t1
sw $t2, 32($s7)

```

2-10

$$f = \&A[0] + \&A[0]$$

2-14



2-16

op=0, rs=3, rt=2, rd=3, shamt=0, func=34
 R-type \$v1 \$v0 \$v1 sub

type = R-type

MIPS: sub \$v1, \$v1, \$v0

Binary: $\frac{0000\ 00}{op} \frac{0001}{rs} \frac{0001}{rt} \frac{0001}{rd} \frac{00000}{shamt} \frac{10010}{func}$

2-18

已思考 48s >

這題在考你：如果把 MIPS 的「通用暫存器數」擴到 128 個（= 7 位元索引），而且指令數量變成原本的 4 倍（= 主要 opcode 需要多 2 位元），在仍維持 32-bit 固定長度指令下，各種指令格式（R/I/J）會被擠壓成什麼樣子、會出現哪些問題。

關鍵數字

- 128 regs $\Rightarrow$ 每個暫存器欄位要 7 bits（原本 5）。
- 指令集合 $\times 4 \Rightarrow$ 主要 opcode 由 6 bits 變 8 bits。
- 指令長度固定 32 bits。

在這些假設下的推導與結論

1. I-type (opcode | rs | rt | imm)

- 位數 = $32 - (8 + 7 + 7) = 10$ bits 的 immediate。
- 影響：beq/bne 的位移是 signed 10 位元、再左移 2（word 對齊），可跳範圍為 ± 512 指令（約 ± 2 KB），比原本 16 位元的 ± 32 KB 小很多。

2. J-type (opcode | target)

- 位數 = $32 - 8 = 24$ bits 的目標欄位。
- 影響：有效位址仍取 PC[31:28] | target << 2，變成 26 位元有效位移，可跳的區域由原本的 256 MB 縮成 64 MB（縮 4 倍）。

3. R-type (傳統 opcode(=0) | rs | rt | rd | shamt | funct)

- 需要的位數 = 8（新 opcode 欄寬）+ 7×3 （rs/rt/rd）+ 5（shamt）+ 6（funct）= 40 bits。
- 結論：塞不下！也就是說，在 32-bit 固定長度下，同時把暫存器擴到 128 且 opcode 擴到 8 bits，原本的三暫存器 R-format 無法維持。

可行的設計方向（要選其一或混搭）

- 改 R-type 為 兩運算元格式（例如讓 rd=rt 隱含）： $8 + 7 + 7 = 22$ ，剩餘 10 bits 可給 funct/shamt。
- 移除/合併 funct 與 shamt：多數運算用不同 opcode 編碼，shamt 僅保留在移位指令的專用格式。
- 分裂更多專用格式（像移位/乘除各自有格式），把稀有欄位搬到專用格式。
- 放寬條件：改變指令長度（>32 bits）或不把 opcode 擴到 8 bits（就不能達到 $\times 4$ 指令數），或使用前綴/延伸碼。

總結：題目要你看清欄位寬的取捨。在維持 32-bit 的前提下，

- I-type 的立即數縮成 10 bits，
- J-type 的目標縮成 24 bits（有效跳躍區 64 MB），
- 傳統 R-type 40 bits 超額 $\rightarrow$ 必須改格式或改其他假設。

